

Intro To Routing

Routing in Laravel is a way to define the endpoints or routes of your application to direct incoming HTTP requests. Meaning that when a user visits a URL, the application should know what to do with it. Usually, a route is connected to some controller method. We'll get into controllers soon, but it doesn't have to load a controller method. You can load a view directly or just return something directly from a function.

Basic Routing

Laravel provides a clean and simple way to define routes. Your route files are located in the `/routes` directory. The main route file is called `web.php`. These files are automatically loaded by Laravel using the configuration in the `bootstrap/app.php` file.

Open the `routes/web.php` file. You should see something like the following:

At the very top of the file, you will see something like this:

```
use Illuminate\Support\Facades\Route;
```

This line is importing the `Route` facade which is used to define routes. A facade is a class that provides a static interface to an underlying class. In this case, the `Route` facade provides a static interface to the `Route` class.

You'll see a lot of imports that gave "illuminate". This is because Laravel's creator, Taylor Otwell, named the framework's core components "Illuminate" to convey the idea of providing clarity and illumination to the process of building web applications.

Below the import statement, you will see a route definition like this:

```
Route::get('/', function () {  
    return view('welcome');  
});
```

Here, we are using the `Route` facade to define a route. The `get` method is used to define a route that responds to HTTP GET requests. The first argument is the URI that the route responds to. In this case, the route responds to the root URI `/`. The second argument is a closure, which is an inline function. In this case, the route function is returning a view names `welcome`. That `welcome` view is located in the `resources/views` directory and is a blade template, which we will also be getting into soon. This is what we see when we visit the root URL of the application.

Returning Values

This particular route is displaying a view, but that isn't the only thing you can do with routes. In many cases, your routes will direct to a controller, which we'll go over in a little bit.

We can just return a string from a route. For example:

```
Route::get('/jobs', function () {  
    return 'Available Jobs';  
});
```

You can also embed HTML in the string:

```
Route::get('/jobs', function () {  
    return '<h1>Available Jobs</h1>';  
});
```

Now when you visit <http://localhost:8000/jobs>, you will see the string **Available Jobs**.

Other HTTP Methods

If you wanted to have a route respond to a POST request, you would use the **post** method. For example:

```
Route::post('/submit', function () {  
    return 'Submitted!';  
});
```

You can test it out using any HTTP client. Here is a CURL request to test it:

```
curl -X POST http://localhost:8000/submit
```

If you get a 419 error, it's because Laravel has built in protection against cross site request forgery (CSRF). Normally you would submit a form to a POST route with a CSRF token. However, if you want to make an exception for specific urls, you can open the [bootstrap/app.php](#) file and add the following:

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->validateCsrfTokens(except: [  
        '/submit'  
    ]);  
})
```

Now you can make the request from another domain.

There are all kinds of methods you can use to define routes. Here are a few examples:

```
Route::get($uri, $callback);  
Route::post($uri, $callback);  
Route::put($uri, $callback);
```

```
Route::patch($uri, $callback);
Route::delete($uri, $callback);
Route::options($uri, $callback);
```

Multiple HTTP Verbs

You can also define a route that responds to multiple HTTP verbs. For example:

```
Route::match(['get', 'post'], '/submit', function () {
    return 'Submitted!';
});
```

If you wanted to respond to all HTTP verbs, you could use the `any` method:

```
Route::any('/submit', function () {
    return 'Submitted!';
});
```

Named Routes

You can also give your routes names. This is useful for generating URLs in your views. For example:

```
Route::get('/jobs', function () {
    return 'Available Jobs';
})->name('jobs');
```

Now create a route at `/test` with a link to the `jobs` route:

```
Route::get('/test', function () {
    $url = route('jobs');
    return "<a href='$url'>Click here</a>";
});
```

Now when you visit `http://localhost:8000/test`, you will see a link to the `jobs` route.

Returning JSON

Laravel is also great for building JSON APIs. In Laravel routes you can actually return an array and Laravel will automatically convert it to JSON. For example:

```
Route::get('/api/users', function () {  
    return [  
        'name' => 'John Doe',  
        'email' => 'john@gmail.com',  
    ];  
});
```

Now when you visit <http://localhost:8000/api/users>, you will see the JSON response:

```
{  
  "name": "John Doe",  
  "email": "
```

If you are building an API though, you should consider using Laravel's API routes and Sanctum for authentication. We will go over that in a later chapter.

Redirect Routes

You can also define routes that redirect to another URL. For example:

```
Route::redirect('/here', '/there');
```

Listing Routes

You can also list all the routes in your application by running the following command:

```
php artisan route:list
```

This will show you a list of all the routes in your application, along with the HTTP method, URI, name, and action.

Remove everything except the original code from the [routes/web.php](#) file. In the next lesson, we will go over route parameters.